

Descripción Detallada y Analítica

El diseño de clases de POO SmartHome aplica los principios de Programación Orientada a Objetos, con énfasis en el encapsulamiento.

Este enfoque protege los atributos sensibles y limita su acceso a través de métodos controlados, garantizando seguridad, consistencia y modularidad—aspectos fundamentales en un sistema de hogar inteligente.

Entidades Principales Diagramadas

- User
- AdminUser
- StandardUser
- Dispositivo
- Automation
- AuthSystem

Abstracción

La abstracción permite representar las entidades reales del sistema (usuarios, dispositivos, autenticación), simplificando su implementación interna.

- User → Representa cualquier persona registrada en el sistema, ocultando cómo se gestionan sus datos.
- Dispositivo → Modela un artefacto de hogar inteligente con atributos (tipo, marca, modelo, estado, modo) y operaciones (encender(), apagar(), consultarEstado()).
- AuthSystem → Abstrae el mecanismo de registro, autenticación e inicio de sesión.

Herencia

Se aplica herencia en la jerarquía de usuarios:

- User → clase padre.
- AdminUser y StandardUser → heredan atributos y métodos de Usuario, agregando funcionalidades específicas:
 - AdminUser → gestionarSistema()
 - StandardUser → consultarDispositivo(), ejecutarAutomatizacion()
Esto evita duplicación de código y asigna responsabilidades diferenciadas

Composición y Agregación

Las relaciones entre las clases se definen así:

- Usuario ↔ Dispositivo → Asociación: un usuario puede interactuar con múltiples dispositivos.
- Automation ↔ Dispositivo → Composición: la automatización depende de los dispositivos para funcionar.
- AuthSystem ↔ User → Agregación: gestiona múltiples usuarios, pero no depende de cada

Encapsulación aplicada en las entidades:

User

Atributos privados (-): nombre, apellido, mail, contraseña, is_admin. La contraseña, dato sensible, no permite acceso directo, mientras que is_admin define privilegios protegidos. Métodos públicos (+): registrarse(), iniciarSesion(). Este encapsulamiento asegura que los usuarios solo puedan crearse o autenticarse mediante operaciones seguras.

AdminUser

Hereda de Usuario, mantiene los atributos privados y expone solo un método adicional: `gestionarSistema()`. La herencia respeta el encapsulamiento, permitiendo extender Usuario sin comprometer su seguridad interna.

StandardUser

Hereda de Usuario, conserva los atributos encapsulados y expone `consultarDispositivo()` y `ejecutarAutomatizacion()`. El encapsulamiento impide que un usuario estándar manipule directamente atributos sensibles.

Dispositivo

Atributos privados (-): tipo, marca, modelo, estado, modo. Estado y modo no pueden modificarse libremente. Métodos públicos (+): `encender()`, `apagar()`, `consultarEstado()`. El encapsulamiento asegura que los cambios en dispositivos ocurran de forma controlada.

Automation

Encapsula reglas mediante `automatizarAire(temp)` y `automatizarLuz(presencia)`. Actúa como intermediario seguro, evitando el acceso directo a los atributos de Dispositivo.

AuthSystem

Métodos públicos (+): `register()`, `authentication()`, `userSession()`. El encapsulamiento evita la manipulación directa de la lista de usuarios, requiriendo que todas las operaciones se realicen mediante métodos controlados.

relaciones desde la óptica del encapsulamiento

Usuario ..> Dispositivo: el usuario utiliza solo métodos públicos, sin acceder a

atributos.

Automation ..> Dispositivo: controla dispositivos sin exponer sus atributos internos.

Usuario → AuthSystem: interactúa únicamente mediante métodos públicos de autenticación.

AuthSystem ..> Usuario: gestiona usuarios encapsulados sin exponer sus datos sensibles.

Beneficios del encapsulamiento en este diseño

Seguridad: protege contraseñas y estados.

Consistencia: asegura cambios válidos en el sistema.

Mantenibilidad: los cambios internos no afectan el uso externo.

Claridad de roles: cada clase expone solo lo necesario.